

Chapter Overview

Chapter 02 covers the full Gang of Four (GoF) design patterns catalogue as applied in the ShopVerse codebase. All 23 patterns are categorised into Creational, Structural, and Behavioural, with code examples drawn directly from ShopVerse classes, anti-pattern comparisons, UML-style text diagrams, and a pattern-selection decision tree.

Category	Patterns	ShopVerse Count
Creational (5)	Singleton, Factory Method, Abstract Factory, Builder, Prototype	8 usages
Structural (7)	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy	11 usages
Behavioural (11)	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor, Null	14 usages

Singleton

Aspect	Details
Intent	Ensure a class has exactly one instance; provide global access
ShopVerse Usage	ApplicationConfig, DatabaseConnectionPool, CacheManager
Implementation	Spring @Bean with default singleton scope (default); no hand-rolled Singletons needed
Thread-Safety	Spring manages one instance per ApplicationContext – thread-safe by design
Anti-Pattern	Hand-rolled double-checked locking with static volatile field – verbose; Spring removes this need

Singleton Pattern Structure

private static volatile instance (pre-Spring approach)

private constructor prevents external instantiation

static getInstance() { if null → synchronized block → create }

Spring Equivalent (preferred)

@Configuration public class AppConfig { @Bean ApplicationConfig config() {

Spring guarantees one bean per context – no volatile field needed

```
// Spring Singleton (ShopVerse pattern – preferred)
@Service public class OrderService { // Spring creates ONE instance
private final OrderRepository repo; // injected – immutable after construction
// No mutable static state → inherently thread-safe
}
// Hand-rolled Singleton (avoid – shown for comparison only)
public class LegacyConfig {
private static volatile LegacyConfig INSTANCE;
public static LegacyConfig get() {
if (INSTANCE == null) synchronized(LegacyConfig.class) {
if (INSTANCE == null) INSTANCE = new LegacyConfig(); }
return INSTANCE; } }
```

Factory Method

Aspect	Details
Intent	Define interface for creating objects; subclasses decide which class to instantiate
ShopVerse Usage	NotificationFactory → EmailNotification / SMSNotification / PushNotification
Key Classes	NotificationFactory (abstract), EmailNotificationFactory (concrete), NotificationService (client)
Benefit	New notification type = new factory subclass; no change to NotificationService

```
// Abstract creator
public abstract class NotificationFactory {
public abstract Notification createNotification(NotificationRequest req);
public void send(NotificationRequest req) {
Notification n = createNotification(req); // factory method
n.prepare(); n.send(); n.log(); }
}
public class EmailNotificationFactory extends NotificationFactory {
```

```

@Override public Notification createNotification(NotificationRequest req) {
    return new EmailNotification(req.getTo(), req.getSubject(), req.getBody()); } }
public class SMSNotificationFactory extends NotificationFactory {
    @Override public Notification createNotification(NotificationRequest req) {
        return new SMSNotification(req.getTo(), req.getBody()); } }

```

Abstract Factory

Aspect	Details
Intent	Create families of related objects without specifying their concrete classes
ShopVerse Usage	PaymentGatewayFactory → { StripeGateway, StripeWebhookVerifier, StripeRefundProcessor }
vs Factory Method	Factory Method creates one product; Abstract Factory creates a FAMILY of products
Benefit	Swap the entire payment provider (Stripe → PayPal) by changing one factory instance

```

public interface PaymentGatewayFactory {
    PaymentProcessor createProcessor();
    WebhookVerifier createWebhookVerifier();
    RefundProcessor createRefundProcessor();
}

public class StripeGatewayFactory implements PaymentGatewayFactory {
    public PaymentProcessor createProcessor() { return new StripeProcessor(apiKey); }
    public WebhookVerifier createWebhookVerifier(){ return new StripeWebhookVerifier(secret); }
    public RefundProcessor createRefundProcessor(){ return new StripeRefundProcessor(apiKey); }
}

```

Builder

Aspect	Details
Intent	Construct complex objects step-by-step; same process → different representations
ShopVerse Usage	Order.builder(), OrderSearchCriteria.builder(), ApiResponse.builder()
Lombok @Builder	Generates fluent builder with all fields + build() method automatically
Mandatory Fields	Use @Builder with @NonNull or validate in build() step
Immutability	Combine with @Value (@Builder + final fields) for immutable value objects

```

@Builder @Getter public class Order {
    private final long id;
    private final long customerId;
    private final List items;
    private final OrderStatus status;
    private final Instant createdAt;
}

// Usage in OrderService
Order order = Order.builder()
    .customerId(req.customerId())
    .items(mapToItems(req.items()))
    .status(OrderStatus.PENDING)
    .createdAt(Instant.now())
    .build();

```

Prototype

Aspect	Details
Intent	Create new objects by cloning an existing prototype
ShopVerse Usage	ProductTemplate.clone() for new product from template; OrderDraft deep-copy for quote-to-order

Aspect	Details
Java Approach	implement Cloneable + override clone(), OR custom copy constructor, OR BeanUtils.copyProperties
Deep vs Shallow	Shallow: copies references. Deep: copies referenced objects too. ShopVerse uses copy constructors for deep copy

```
public class ProductTemplate implements Cloneable {
    private String name; private BigDecimal basePrice;
    private List tags; // needs deep copy!
    @Override public ProductTemplate clone() {
        try {
            ProductTemplate copy = (ProductTemplate) super.clone();
            copy.tags = new ArrayList<>(this.tags); // deep copy the list
            return copy;
        } catch (CloneNotSupportedException e) { throw new AssertionError(); }
    }
}
```

Adapter

Aspect	Details
Intent	Convert one interface into another that a client expects
ShopVerse Usage	ExternalPaymentGatewayAdapter wraps Stripe SDK to fit PaymentGateway interface
Types	Object Adapter (composition – preferred), Class Adapter (inheritance – avoid in Java)
Benefit	External library changes → only adapter changes; core code unaffected

```
// Target interface (our system expects this)
public interface PaymentGateway {
    PaymentResult charge(long customerId, BigDecimal amount, String currency);
    void refund(String transactionId, BigDecimal amount);
}

// Adaptee (Stripe SDK - we cannot change this)
// StripeClient.createCharge(Map params)
// StripeClient.createRefund(String chargeId, long amountCents)
// Adapter (bridges the gap)
@Component public class StripePaymentAdapter implements PaymentGateway {
    private final StripeClient stripe; // composition - holds adaptee
    public PaymentResult charge(long cid, BigDecimal amt, String cur) {
        Map p = Map.of("amount", amt.multiply(CENTS).longValue(),
            "currency", cur, "customer", cid);
        Charge c = stripe.createCharge(p);
        return new PaymentResult(c.getId(), SUCCESS); }
}
```

Decorator

Aspect	Details
Intent	Add behaviour to an object dynamically without changing its class
ShopVerse Usage	ProductRepository decorated with CachingProductRepository; LoggingOrderRepository
vs Inheritance	Decorator composes at runtime; inheritance composes at compile time
Spring AOP Alternative	@Cacheable/@Around achieve similar but through proxy – Decorator is explicit

```
// Component interface
public interface ProductRepository { Optional findById(long id); }
// Concrete component
@Repository public class JpaProductRepository implements ProductRepository { ... }
// Decorator - adds caching layer
@Primary @Repository
public class CachingProductRepository implements ProductRepository {
    private final ProductRepository delegate; // wraps concrete
    private final Cache cache;
    public Optional findById(long id) {
        return Optional.ofNullable(cache.getIfPresent(id))
            .or(() -> { var r = delegate.findById(id);
                r.ifPresent(p -> cache.put(id, p)); return r; });
    }
}
```

Facade

Aspect	Details
Intent	Provide a unified, simplified interface over a complex subsystem

Aspect	Details
ShopVerse Usage	OrderFacade orchestrates OrderService + InventoryService + PaymentService + NotificationService
Benefit	Controller calls ONE method; internal coordination is hidden
vs Mediator	Facade simplifies access to subsystem. Mediator coordinates BETWEEN peers.

```

@Service public class OrderFacade {
    private final OrderService orders;
    private final InventoryService inventory;
    private final PaymentService payments;
    private final NotificationService notifications;
    // Single entry point - hides cross-service orchestration
    @Transactional
    public OrderResponse placeOrder(PlaceOrderRequest req) {
        inventory.reserve(req.items()); // 1. reserve stock
        Order order = orders.create(req); // 2. persist order
        payments.charge(req.customerId(), order.getTotal()); // 3. charge
        notifications.sendConfirmation(order); // 4. notify
        return OrderMapper.toResponse(order); // 5. return
    }
}

```

Proxy

Aspect	Details
Intent	Provide a surrogate for another object to control access, add caching, or add logging
Types	Virtual Proxy (lazy init), Remote Proxy (RPC), Protection Proxy (auth), Cache Proxy
ShopVerse Usage	Spring AOP creates JDK Dynamic Proxies for @Transactional, @Cacheable, @Async, @Secured
JDK Dynamic Proxy	Works on interfaces; Spring default when interface present
CGLIB Proxy	Subclass-based; Spring uses when no interface; @SpringBootApplication enables

```

// Spring creates proxy automatically for @Transactional
@Service public class PaymentService {
    @Transactional // Spring wraps this method in a TransactionInterceptor proxy
    public void processPayment(long orderId) {
        // Spring proxy: begin tx → call this → commit / rollback
    }
}

// Manual cache proxy for ProductService
@Cacheable(value="products", key="#id")
public Optional findById(long id) { return repo.findById(id); }

```

■ Self-invocation bypasses Spring AOP proxies! @Transactional and @Cacheable on a method called from within the same class won't work unless you inject a self-reference or use AspectJ mode.

Flyweight

Aspect	Details
Intent	Share common intrinsic state among many fine-grained objects to save memory
ShopVerse Usage	ProductCategory flyweight pool – shared Category objects instead of duplicating per product
Intrinsic State	Shared, immutable – category name, id, parentId
Extrinsic State	Per-product – categoryId reference (not the full object)
Java Built-in	String intern pool, Integer cache (-128 to 127), Boolean.TRUE/FALSE

```

// Flyweight factory - ensures shared instances

```

```

@Component public class CategoryFlyweightFactory {
    private final Map pool = new ConcurrentHashMap<>();
    public Category get(long id) {
        return pool.computeIfAbsent(id, k -> categoryRepo.findById(k).orElseThrow());
    }
}
// Product holds only the ID (extrinsic reference)
@Column(name="category_id") private long categoryId;
// When needed: Category cat = flyweightFactory.get(product.getCategoryId());

```

Composite

Aspect	Details
Intent	Compose objects into tree structures; treat individual and composite uniformly
ShopVerse Usage	Category tree – leaf categories and composite parent categories both implement CategoryNode
Key Interface	CategoryNode with totalProductCount() method
Leaf	Concrete category with no children – returns own productCount
Composite	Category with children – returns sum of children.totalProductCount()

```

public interface CategoryNode {
    String getName();
    int totalProductCount();
}
public class LeafCategory implements CategoryNode {
    private final String name; private final int productCount;
    public int totalProductCount() { return productCount; }
}
public class CompositeCategory implements CategoryNode {
    private final String name;
    private final List children = new ArrayList<>();
    public int totalProductCount() {
        return children.stream().mapToInt(CategoryNode::totalProductCount).sum(); }
}

```

Bridge

Aspect	Details
Intent	Decouple abstraction from implementation so both can vary independently
ShopVerse Usage	NotificationSender (abstraction) + NotificationChannel impl (Email/SMS/Push)
vs Adapter	Adapter fixes incompatible interfaces. Bridge designs for future variation.
Benefit	Add new notification type OR new channel without changing the other dimension

```

// Implementation axis
public interface NotificationChannel { void deliver(String recipient, String message); }
public class EmailChannel implements NotificationChannel { ... }
public class SMSChannel implements NotificationChannel { ... }
// Abstraction axis
public abstract class NotificationSender {
    protected final NotificationChannel channel; // bridge to impl
    public NotificationSender(NotificationChannel ch) { this.channel = ch; }
    public abstract void send(Notification n);
}
public class UrgentNotificationSender extends NotificationSender {
    public void send(Notification n) { channel.deliver(n.to(), "[URGENT] " + n.body()); }
}

```

Strategy

Aspect	Details
Intent	Define a family of algorithms, encapsulate each one, make them interchangeable
ShopVerse Usage	PricingStrategy → StandardPricing / DiscountPricing / FlashSalePricing / MemberPricing
Spring Integration	Strategies as Spring @Component beans; selected by CustomerTier enum
Benefit	Add new pricing rule without touching OrderService – open for extension, closed for modification

```
@FunctionalInterface public interface PricingStrategy {
    BigDecimal calculate(Product product, int qty, Customer customer);
}

@Component("STANDARD") public class StandardPricingStrategy implements PricingStrategy {
    public BigDecimal calculate(Product p, int qty, Customer c) {
        return p.getBasePrice().multiply(BigDecimal.valueOf(qty));
    }
}

@Component("VIP") public class MemberPricingStrategy implements PricingStrategy {
    public BigDecimal calculate(Product p, int qty, Customer c) {
        BigDecimal discount = c.getTier().getDiscount(); // e.g. 0.15
        return p.getBasePrice().multiply(ONE.subtract(discount)).multiply(valueOf(qty));
    }
}

// Selector in OrderService
private final Map strategies; // injected by Spring
PricingStrategy ps = strategies.get(customer.getTier().name());
```

Observer

Aspect	Details
Intent	Define one-to-many dependency; when subject changes, all dependents notified automatically
ShopVerse Usage	OrderEventPublisher (subject) → [InventoryListener, NotificationListener, AuditListener, AnalyticsListener]
Spring Events	ApplicationEventPublisher + @EventListener – decoupled, synchronous or async
Kafka Integration	For cross-service: OrderPlacedEvent published to Kafka topic → consumed by other microservices

```
// Spring application events (in-process Observer)
@Component public class OrderEventPublisher {
    private final ApplicationEventPublisher pub;
    public void publishOrderPlaced(Order o) {
        pub.publishEvent(new OrderPlacedEvent(o.getId(), o.getCustomerId(), Instant.now()));
    }
}

@Component public class InventoryListener {
    @EventListener @Async // async = non-blocking
    public void on(OrderPlacedEvent e) { inventory.deduct(e.orderId()); }
}

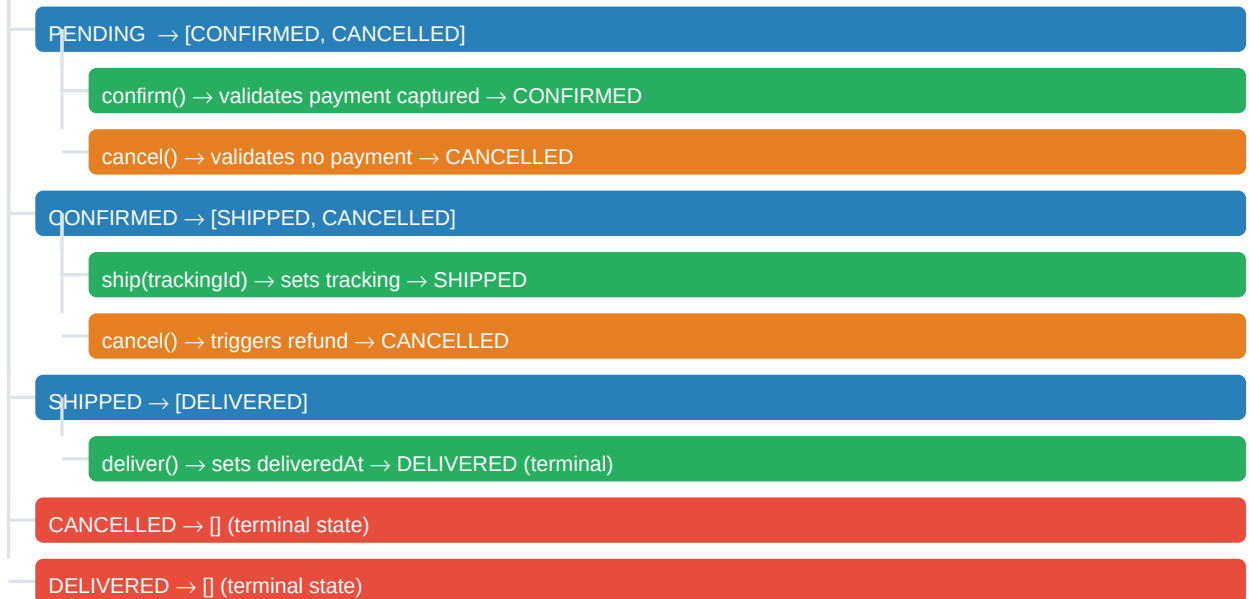
@Component public class NotificationListener {
    @EventListener @Async
    public void on(OrderPlacedEvent e) { notify.sendConfirmation(e.orderId()); }
}
```

State

Aspect	Details
Intent	Allow an object to change behaviour when its internal state changes
ShopVerse Usage	OrderStateMachine – PENDING → CONFIRMED → SHIPPED → DELIVERED / CANCELLED

Aspect	Details
Sealed Interface Approach	OrderState sealed interface; each state validates and performs transitions
Benefit	Invalid transitions throw checked exception; no giant if/switch in OrderService

OrderStateMachine transitions



```

public sealed interface OrderState permits PendingState, ConfirmedState, ShippedState,
DeliveredState, CancelledState {
    OrderState confirm(Order o);
    OrderState ship(Order o, String tracking);
    OrderState deliver(Order o);
    OrderState cancel(Order o);
}

public record PendingState() implements OrderState {
    public OrderState confirm(Order o) { o.setStatus(CONFIRMED); return new ConfirmedState(); }
    public OrderState ship(Order o, String t) { throw new
InvalidTransitionException("PENDING→SHIPPED"); }
    // ...
}

```

Template Method

Aspect	Details
Intent	Define the skeleton of an algorithm; defer some steps to subclasses
ShopVerse Usage	AbstractOrderProcessor defines processOrder() steps; concrete subclasses override
Hook methods	Optional override points (do nothing by default)
Hollywood Principle	Don't call us; we'll call you – parent calls child methods

```

public abstract class AbstractOrderProcessor {
    // Template method - final to prevent override
    public final OrderResult processOrder(Order order) {
        validateOrder(order); // step 1 - abstract
        reserveInventory(order); // step 2 - abstract
        chargePayment(order); // step 3 - abstract
        onOrderProcessed(order); // step 4 - hook (optional)
        return buildResult(order); // step 5 - abstract
    }
}

```

```

}
protected abstract void validateOrder(Order o);
protected abstract void reserveInventory(Order o);
protected abstract void chargePayment(Order o);
protected void onOrderProcessed(Order o) {} // hook - empty default
protected abstract OrderResult buildResult(Order o);
}

```

Chain of Responsibility

Aspect	Details
Intent	Pass requests along a chain of handlers; each decides to handle or pass on
ShopVerse Usage	OrderValidationChain: StockValidator → PriceValidator → FraudValidator → CouponValidator
Spring Integration	Filter chain (Servlet filters), Spring Security filter chain, @ControllerAdvice exception chain
Benefit	Add/remove/reorder validators without changing caller

```

public abstract class OrderValidator {
    private OrderValidator next;
    public OrderValidator setNext(OrderValidator n) { this.next = n; return n; }
    public final ValidationResult validate(Order o) {
        ValidationResult r = doValidate(o);
        return r.isValid() && next != null ? next.validate(o) : r;
    }
    protected abstract ValidationResult doValidate(Order o);
}
// Wiring in OrderService
StockValidator stock = new StockValidator();
stock.setNext(new PriceValidator()).setNext(new FraudValidator()).setNext(new
CouponValidator());
ValidationResult result = stock.validate(order);

```

Command

Aspect	Details
Intent	Encapsulate a request as an object; support undo, queue, log operations
ShopVerse Usage	OrderCommand (place/cancel/refund); AsyncOrderCommand for deferred execution via queue
Undo Support	Command stores state needed to reverse operation; execute() + undo()
Kafka Integration	Commands serialised as Kafka messages → consumers execute asynchronously

```

public interface OrderCommand {
    OrderResult execute();
    void undo(); // rollback the operation
}
public class PlaceOrderCommand implements OrderCommand {
    private final PlaceOrderRequest req;
    private Order createdOrder; // stored for undo
    public OrderResult execute() {
        createdOrder = orderService.create(req);
        return OrderResult.success(createdOrder.getId()); }
    public void undo() {
        if (createdOrder != null) orderService.cancel(createdOrder.getId()); }
}

```

Iterator + Mediator + Memento (Summary)

Pattern	Intent	ShopVerse Usage
Iterator	Traverse collection without exposing internals	PagedIterator for database cursor pagination; ProductCatalogIterator (la
Mediator	Define object that encapsulates how objects interact	OrderEventBus as mediator; Spring ApplicationContext mediates bean c
Memento	Capture & restore object state without violating encapsulation	OrderDraft.saveSnapshot() / restoreSnapshot() for multi-step checkout m

```
// Memento example (OrderDraft)
public class OrderDraft {
    private List items; private String couponCode; private Address address;
    public DraftSnapshot save() { return new DraftSnapshot(new ArrayList<>(items), couponCode,
        address); }
    public void restore(DraftSnapshot snap) { items=new ArrayList<>(snap.items());
        couponCode=snap.coupon(); address=snap.addr(); }
}
public record DraftSnapshot(List items, String coupon, Address addr) {}
```

Creational Pattern Comparison

Pattern	Creates	Key Difference	When to Use
Singleton	1 instance	Only one ever exists	Shared config, connection pools
Factory Method	1 product via subclass	Subclass decides type	Notification type varies by context
Abstract Factory	Family of products	Multiple related types	Swap provider (Stripe ↔ PayPal)
Builder	1 complex object step-by-step	Many optional params; immutability	Order, Query, Config objects
Prototype	Clone of existing	Create from template	OrderDraft → Order; ProductTemplate

Structural Pattern Comparison

Pattern	Purpose	Changes Interface?	Wraps Object?	ShopVerse Class
Adapter	Interface conversion	Yes (new interface)	Yes	StripePaymentAdapter
Decorator	Add behaviour dynamically	No (same interface)	Yes	CachingProductRepository
Facade	Simplify subsystem	Yes (simplified)	Yes (subsystem)	OrderFacade
Proxy	Control access	No (same interface)	Yes	Spring @Transactional proxy
Bridge	Decouple abstraction/impl	Two hierarchies	Yes	NotificationSender + Channel
Composite	Tree structures	No (uniform interface)	Children	CategoryNode tree
Flyweight	Share intrinsic state	No	Shared pool	CategoryFlyweightFactory

Behavioural Pattern Comparison

Pattern	Encapsulates	Key Mechanism	ShopVerse Class
Strategy	Algorithm variation	Composition + polymorphism	PricingStrategy (Standard/VIP/Flash)
Observer	Event notification	Event bus / listener	OrderEventPublisher + Listeners
State	Object state machine	State objects + transitions	OrderStateMachine
Template Method	Algorithm skeleton	Inheritance + hooks	AbstractOrderProcessor
Chain of Responsibility	Request pipeline	Linked handlers	OrderValidationChain
Command	Encapsulated request + undo	Command object	PlaceOrderCommand
Iterator	Traversal of collection	hasNext/next contract	PagedIterator
Mediator	Object interaction coordination	Central hub	OrderEventBus
Memento	State snapshot/restore	Opaque memento object	OrderDraft.DraftSnapshot

Creational: "How should I create objects?"

Need to create objects

- Only ONE instance should ever exist → Singleton (@Bean in Spring)
- Need to CREATE different TYPES of objects
 - Subclass decides which type → Factory Method
 - Need a FAMILY of related types → Abstract Factory
- Object has MANY optional fields / must be immutable → Builder (@Builder)
- Create from an EXISTING object template → Prototype (clone / copy construct)

Structural: "How should objects connect?"

Need to connect objects

- INCOMPATIBLE interfaces → Adapter
- Add BEHAVIOUR without modifying class → Decorator
- Simplify complex SUBSYSTEM → Facade
- CONTROL ACCESS / add caching / logging → Proxy
- TWO varying dimensions → Bridge
- TREE hierarchy of uniform objects → Composite
- SHARE intrinsic state among many objects → Flyweight

Behavioural: "How should objects communicate?"

Need to define object communication

- Interchangeable ALGORITHMS → Strategy
- ONE-TO-MANY event notification → Observer / Spring Events
- Object CHANGES BEHAVIOUR based on state → State
- Algorithm has fixed SKELETON with varying STEPS → Template Method
- Request through PIPELINE of handlers → Chain of Responsibility
- Encapsulate REQUEST as object + support UNDO → Command
- Traverse WITHOUT exposing structure → Iterator

Principle	Meaning	Violation Example	ShopVerse Compliance
S – Single Responsibility	One class, one reason to change	OrderService that also sends emails	OrderService creates ANOrder, CancelOrder, NotificationService
O – Open for Extension, closed for modification	Open for extension, closed for modification	NotificationService (type) in NotificationService	Add SMSNotificationFactory; don't touch NotificationService
L – Liskov Substitution	Subtypes replaceable for base type	PremiumOrder.cancel() silently ignores state	All OrderState implementors honour the OrderState contract
I – Interface Segregation	Small focused interfaces instead of large ones	RepositoryEs with 30 methods	ReadOnlyProductRepository, WritableProductRepository
D – Dependency Inversion	Depend on abstractions, not concrete classes	OrderService creates new StripeOrder	OrderService depends on PaymentGateway interface; StripeAdapter

■ Do

- ✓ Prefer Strategy over switch/if-else chains for algorithm selection.
- ✓ Use Spring `@EventListener` for Observer – avoids manual listener registration.
- ✓ Apply Builder for objects with > 4 constructor parameters.
- ✓ Use Facade to simplify controller-to-service orchestration.
- ✓ Implement State pattern for entity lifecycle to prevent invalid transitions.
- ✓ Prefer Chain of Responsibility for validation pipelines – each validator single-responsibility.
- ✓ Apply Abstract Factory when you need to swap entire provider families.
- ✓ Document pattern intent with a `// Pattern:` comment above the class.

■ Anti-Patterns

- Singleton with mutable static state – thread safety nightmare.
- Overusing Factory for simple `"new Foo()"` – adds indirection without value.
- Deep inheritance chains instead of Decorator – fragile base class problem.
- God class that implements multiple patterns – becomes unmaintainable.
- Pattern matching every problem: "I have a hammer, everything is a nail."
- Forgetting to call `super.clone()` in Prototype – misses field copying.
- Observer without `@Async` – synchronous listeners block the publisher thread.
- State pattern with mutable State objects – should be immutable records or singletons.

Pattern	Category	ShopVerse Class	Key Participants
Singleton	Creational	ApplicationConfig, CacheManager	Single instance via Spring @Bean
Factory Method	Creational	NotificationFactory	Creator, ConcreteCreator, Product
Abstract Factory	Creational	PaymentGatewayFactory	AbstractFactory, ConcreteFactory, ProductFamily
Builder	Creational	Order, OrderSearchCriteria	Builder, ConcreteBuilder, Director, Product
Prototype	Creational	ProductTemplate, OrderDraft	Prototype, clone(), deep copy
Adapter	Structural	StripePaymentAdapter	Target, Adaptee, Adapter (composition)
Decorator	Structural	CachingProductRepository	Component, ConcreteComponent, Decorator
Facade	Structural	OrderFacade	Facade, Subsystem classes
Proxy	Structural	Spring AOP (@Transactional)	Subject, RealSubject, Proxy
Bridge	Structural	NotificationSender + Channel	Abstraction, Implementation, RefinedAbstraction
Composite	Structural	CategoryNode tree	Component, Leaf, Composite
Flyweight	Structural	CategoryFlyweightFactory	Flyweight, FlyweightFactory, extrinsic state
Strategy	Behavioural	PricingStrategy	Strategy, ConcreteStrategy, Context
Observer	Behavioural	OrderEventPublisher + Listener	Subject, Observer, ConcreteObserver
State	Behavioural	OrderStateMachine	State, ConcreteState, Context
Template Method	Behavioural	AbstractOrderProcessor	AbstractClass, ConcreteClass, hook
Chain of Resp.	Behavioural	OrderValidationChain	Handler, ConcreteHandler, Client
Command	Behavioural	PlaceOrderCommand	Command, ConcreteCommand, Invoker, Receiver
Iterator	Behavioural	PagedIterator	Iterator, ConcreteIterator, Aggregate
Mediator	Behavioural	OrderEventBus	Mediator, ConcreteMediator, Colleague
Memento	Behavioural	OrderDraft.DraftSnapshot	Originator, Memento, Caretaker

Interview Quick-Check

Question	Concise Answer
Strategy vs Template Method?	Strategy: algorithm varies via composition. Template Method: skeleton fixed in parent; steps deferred to subclasses.
Decorator vs Proxy?	Both wrap objects. Decorator adds behaviour (same interface). Proxy controls access (same interface).
Factory Method vs Abstract Factory?	FM: one product type; subclass decides concrete class. AF: family of products; concrete factory creates concrete products.
When not to use a pattern?	When the problem is simple enough that the pattern adds indirection without benefit. YAGNI principle.
Observer vs Mediator?	Observer: 1-to-many push notification. Mediator: N-to-N; all communication goes through mediator.